JAVASCRIPT

- JavaScript is the world's most popular programming language.
- JavaScript is the programming language of the Web.
- JavaScript is easy to learn.
- JavaScript was invented by Brendan Eich in 1995, and became an ECMA standard in 1997.
- ECMA-262 is the official name of the standard. ECMAScript is the official name of the language.

Why Study JavaScript?

JavaScript is one of the **3 languages** all web developers **must** learn:

1. **HTML** to define the content of web pages
2. **CSS** to specify the layout of web pages
3. **JavaScript** to program the behavior of web pages

- **JAVASCRIPT CAN CHANGE HTML CONTENT**

One of many JavaScript HTML methods is `getElementById()`.

- **JAVASCRIPT CAN CHANGE HTML ATTRIBUTE VALUES**
- **JAVASCRIPT CAN CHANGE HTML STYLES (CSS)**

Changing the style of an HTML element, is a variant of changing an HTML attribute:

- **JAVASCRIPT CAN SHOW HTML ELEMENTS**

Showing hidden HTML elements can also be done by changing the `display` style

THE <SCRIPT> TAG

In HTML, JavaScript code is inserted between `<script>` and `</script>` tags.

```
<html>

<head>

<title>my web page</title>

</head>

<body>

<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>

</body>

</html>
```

EXTERNAL JAVASCRIPT

Scripts can also be placed in external files:

External scripts are practical when the same code is used in many different web pages.

JavaScript files have the file extension **.js**.

To use an external script, put the name of the script file in the `src` (source) attribute of a `<script>` tag:

```
<script src="myScript.js"></script>
```

EXTERNAL JAVASCRIPT ADVANTAGES

Placing scripts in external files has some advantages:

- It separates HTML and code
- It makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files can speed up page loads

To add several script files to one page - use several script tags:

JAVASCRIPT OUTPUT

JavaScript Display Possibilities

JavaScript can "display" data in different ways:

- Writing into an HTML element, using `innerHTML`.
- Writing into the HTML output using `document.write()`.
- Writing into an alert box, using `window.alert()`.
- Writing into the browser console, using `console.log()`.

USING INNERHTML

To access an HTML element, JavaScript can use the `document.getElementById(id)` method.

The `id` attribute defines the HTML element. The `innerHTML` property defines the HTML content:

USING DOCUMENT.WRITE()

For testing purposes, it is convenient to use `document.write()`:

USING WINDOW.ALERT()

You can use an alert box to display data:

USING CONSOLE.LOG()

For debugging purposes, you can call the `console.log()` method in the browser to display data.

JAVA SCRIPTS INPUTS:

WE USE PROMPT FOR GIVING INPUTS.

PARSE FUNCTION:

The main purpose of using the parseInt function is **to extract a number from a string**. This turns the returned value to an actual number. In the example above, 3 is a string and not an actual number.

JAVASCRIPT PROGRAMS

- A **computer program** is a list of "instructions" to be "executed" by a computer.
- In a programming language, these programming instructions are called **statements**.
- A **JavaScript program** is a list of programming **statements**.

JAVASCRIPT STATEMENTS

- JavaScript statements are composed of:
- Values, Operators, Expressions, Keywords, and Comments.

- Most JavaScript programs contain many JavaScript statements.
- The statements are executed, one by one, in the same order as they are written.
- JavaScript programs (and JavaScript statements) are often called JavaScript code.

SEMICOLONS ;

- Semicolons separate JavaScript statements.
- When separated by semicolons, multiple statements on one line are allowed:

JAVASCRIPT KEYWORDS

- JavaScript statements often start with a **keyword** to identify the JavaScript action to be performed.

Keyword	Description
var	Declares a variable
let	Declares a block variable

const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

JAVASCRIPT SYNTAX

JavaScript syntax is the set of rules, how JavaScript programs are constructed:

```
// How to create variables:
var x;
let y;

// How to use variables:
x = 5;
```

```
y = 6;
let z = x + y;
```

JavaScript Values

The JavaScript syntax defines two types of values:

- Fixed values
- Variable values

Fixed values are called **Literals**.

Variable values are called **Variables**.

JAVASCRIPT LITERALS

The two most important syntax rules for fixed values are:

1. **Numbers** are written with or without decimals:

```
10.50
```

```
1001
```

2. **Strings** are text, written within double or single quotes:

```
"John Doe"
```

```
'John Doe'
```

JAVASCRIPT VARIABLES

In a programming language, **variables** are used to **store** data values.

JavaScript uses the keywords `var`, `let` and `const` to **declare** variables.

An **equal sign** is used to **assign values** to variables.

```
let x;
x = 6;
```

In this example, x is defined as a variable. Then, x is assigned (given) the value 6:

JAVASCRIPT OPERATORS

JavaScript uses **arithmetic operators** (+ - * /) to **compute** values:

JavaScript uses an **assignment operator** (=) to **assign** values to variables:

JAVASCRIPT EXPRESSIONS

An expression is a combination of values, variables, and operators, which computes to a value.

The computation is called an evaluation.

- Expressions can also contain variable values:
- The values can be of various types, such as numbers and strings.

JAVASCRIPT KEYWORDS

JavaScript **keywords** are used to identify actions to be performed.

- The `let` keyword tells the browser to create variables:
- The `var` keyword also tells the browser to create variables:

JAVASCRIPT COMMENTS

Not all JavaScript statements are "executed".

Code after double slashes `//` or between `/*` and `*/` is treated as a **comment**.

Comments are ignored, and will not be executed:

JavaScript Identifiers / Names

Identifiers are JavaScript names.

Identifiers are used to name variables and keywords, and functions.

The rules for legal names are the same in most programming languages.

A JavaScript name must begin with:

- A letter (A-Z or a-z)
- A dollar sign ($)
- Or an underscore (_)

Subsequent characters may be letters, digits, underscores, or dollar signs.

NOTE

Numbers are not allowed as the first character in names.

This way JavaScript can easily distinguish identifiers from numbers.

JAVASCRIPT IS CASE SENSITIVE

All JavaScript identifiers are **case sensitive**.

JAVASCRIPT ARITHMETIC OPERATORS

Arithmetic operators are used to perform arithmetic on numbers:

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

JAVASCRIPT ASSIGNMENT OPERATORS

Assignment operators assign values to JavaScript variables.

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

JAVASCRIPT COMPARISON OPERATORS

Operator	Description

==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

JAVASCRIPT LOGICAL OPERATORS

Operator	Description

&&	logical and
||	logical or
!	logical not

JAVASCRIPT IF, ELSE, AND ELSE IF

Conditional statements are used to perform different actions based on different conditions.

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions.

You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- Use `if` to specify a block of code to be executed, if a specified condition is true
- Use `else` to specify a block of code to be executed, if the same condition is false
- Use `else if` to specify a new condition to test, if the first condition is false
- Use `switch` to specify many alternative blocks of code to be executed

THE IF STATEMENT

Use the `if` statement to specify a block of JavaScript code to be executed if a condition is true.

```
if (condition) {
  //  BLOCK OF CODE TO BE EXECUTED IF THE CONDITION IS TRUE
}
```

THE ELSE STATEMENT

Use the `else` statement to specify a block of code to be executed if the condition is false.

```
if (condition) {
  //  BLOCK OF CODE TO BE EXECUTED IF THE CONDITION IS TRUE
} else {
  //  BLOCK OF CODE TO BE EXECUTED IF THE CONDITION IS FALSE
}
```

THE ELSE IF STATEMENT

Use the `else if` statement to specify a new condition if the first condition is false.

```
if (condition1) {
  //  BLOCK OF CODE TO BE EXECUTED IF CONDITION1 IS TRUE
} else if (condition2) {
  //  BLOCK OF CODE TO BE EXECUTED IF THE CONDITION1 IS FALSE AND
CONDITION2 IS TRUE
} else {
  //  BLOCK OF CODE TO BE EXECUTED IF THE CONDITION1 IS FALSE AND
CONDITION2 IS FALSE
}
```

JAVASCRIPT FOR LOOP

Loops can execute a block of code a number of times.

JAVASCRIPT LOOPS

Loops are handy, if you want to run the same code over and over again, each time with a different value.

Different Kinds of Loops

JavaScript supports different kinds of loops:

- `for` - loops through a block of code a number of times
- `for/in` - loops through the properties of an object
- `for/of` - loops through the values of an iterable object
- `while` - loops through a block of code while a specified condition is true
- `do/while` - also loops through a block of code while a specified condition is true

THE FOR LOOP

The `for` loop has the following syntax:

```
for (statement 1; statement 2; statement 3) {
  // code block to be executed
}
```

Statement 1 is executed (one time) before the execution of the code block.

Statement 2 defines the condition for executing the code block.

Statement 3 is executed (every time) after the code block has been executed.

THE WHILE LOOP

The `while` loop loops through a block of code as long as a specified condition is true.

SYNTAX

```
while (CONDITION) {
  // code block to be executed
}
```

THE DO WHILE LOOP

The `do while` loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

SYNTAX

```
do {
  // code block to be executed
}
while (CONDITION);
```